Chapter 8

Software Interrupts

Introduction to Interrupts

- **Definition**: Interrupts are asynchronous events external to the processor.
- **Key Characteristics**:
  - **Asynchronous**: Occur independently of the processor's current operation.
  - **Unpredictable**: Exact timing of occurrence cannot be anticipated.
- **Importance**: Allow the processor to respond to external events.

Types of Interrupts in 8088 Processor

- **1. Hardware Interrupts**:
  - Generated by external hardware.
  - Examples: Pressure sensors, external signals.
- **2. Software Interrupts**:
  - Generated within the processor.
  - Act as an extended far call mechanism.
  - Pushes FLAGS, CS, and IP onto the stack.

Software Interrupt Mechanism

- **INT Instruction**:
  - Syntax: INT n (where n is 0-255).
  - Executes Interrupt Service Routine (ISR).
- **Interrupt Vector Table (IVT)**:
  - Location: Physical memory address 0.
  - Size: 1 KB (256 vectors × 4 bytes each).
  - Format:
    - First 2 bytes: Offset of ISR.
    - Next 2 bytes: Segment of ISR.

Operation of INT Instruction

- Microcode steps:
  - Push FLAGS, CS, and IP onto the stack.
  - Disable interrupts by clearing IF and TF flags.
  - Load CS and IP with ISR address from IVT.

Operation of IRET Instruction

- Microcode steps:
  - Pop IP, CS, and FLAGS from the stack.
  - Restore original state of IF and TF.
  - Return to the interrupted program.

Interrupts Reserved by 8088

1. **INT 0**: Division by zero.
   - Triggered by a DIV instruction if the quotient doesn't fit.
2. **INT 1**: Trap (Single Step Interrupt).
   - Used in debugging.
3. **INT 2**: Non-Maskable Interrupt (NMI).
   - Used for critical failures (e.g., hardware failure).
4. **INT 3**: Debug Interrupt.
   - Single-byte opcode, used in debuggers.
5. **INT 4**: Arithmetic Overflow.
   - Triggered by INTO instruction when the overflow flag is set.

Hardware vs. Software Interrupts

Aspect	Hardware Interrupts	Software Interrupts
Generated by	External hardware	Processor itself
Trigger mechanism	Signal on INT or NMI pin	INT instruction
Interrupt handler	Address in IVT	Address in IVT

Real-Time Interrupts (Overview)

- Used for:
  - **Control Applications**: e.g., closing a valve in response to a sensor.
  - **Critical System Responses**: Handling urgent hardware events.

Flexibility of IVT

- **Mapping Mechanism**:
  - Address of ISR can be changed dynamically.
  - Enables flexible control of interrupt handling.

Summary

- **Key Concepts**:
  - INT and IRET operation.
  - IVT and its structure.
  - Reserved interrupts in 8088.
- **Applications**:
  - Debugging, control systems, and hardware failure management.

BIOS and DOS Interrupts

- **Purpose of Interrupts in IBM PCs:**
  - Enable user programs to access system services for hardware (e.g., floppy drive, VGA, clock).
  - Avoids the need to understand detailed hardware mechanisms.
- **BIOS and Firmware**:
  - **BIOS (Basic Input Output Services)**: Interface to basic hardware.
  - **Firmware**: Software burned into ROM by the manufacturer.
  - BIOS Services: Low-level services for hardware control.

BIOS Role in Boot Process

- **Boot Sequence**:
  - BIOS gains control after power-on at a fixed address.
  - Executes hardware testing and initializes bootstrap to load OS.
- **Layers of Abstraction**:
  - **Hardware** → BIOS Services → OS Services.
  - BIOS provides device-independent services like clearing the screen, displaying characters, etc.

Interrupt Mechanism for BIOS

- **Why Use Interrupts?**
  - Flexible mechanism for accessing firmware services.
  - Allows manufacturers to place routines at any memory location.
- **BIOS Interrupts**:
  - Examples:
    - **INT 10**: Video services.
    - **INT 16**: Keyboard services.
    - **INT 17**: Parallel port services.

DOS Interrupts

- **Single Entry Point**:
  - **INT 21**: Access to all DOS services.
  - Services identified by **Service Numbers** in registers.
- **Service Numbers**:
  - Service number often in AH.
  - Sub-services in AL or BL.

BIOS Interrupt Service Numbers

- **Service Numbering Scheme**:
  - Example: INT 10 Service 0 → Character printing.
  - Allows efficient multiplexing of services within an interrupt.
- **Arguments in Interrupts**:
  - Passed via registers (not the stack).
  - Documented per service and sub-service.

Example: INT 10 Service 13

- **Program Objective**: Print a string using BIOS video services.
- **Why BIOS Works Universally**:
  - Independent of video memory location or hardware specifics.

```
; print string using bios service
[org 0x0100]
        jmp start
message:        db 'Hello World'
start:          mov ah, 0x13        ; service 13 - print string
mov     al, 1                       ; subservice 01 – update cursor
mov     bh, 0                       ; output on page 0
mov     bl, 7                       ; normal attrib
mov     dx, 0x0A03                  ; row 10 column 3
mov     cx, 11                      ; length of string
push    cs
pop     es                          ; segment of string
mov     bp, message                 ; offset of string
int     0x10                        ; call BIOS video service
mov     ax, 0x4c00                  ; terminate program
int     0x21
```